arXiv:2602.13279v2 [cs.SI] 22 Mar 2026

LLM-Enhanced Rumor Detection via Virtual Node Induced Edge Prediction

Jiran Tao
Department of Data Science and Artificial Intelligence
Hong Kong Polytechnic University
Hung Hom, Kowloon, Hong Kong
22117758r@connect.polyu.hk

Cheng Wang
School of Mathematical Sciences
Shanghai Jiao Tong University
Shanghai, China
chengwang@sjtu.edu.cn

Binyan Jiang
Department of Data Science and Artificial Intelligence
Hong Kong Polytechnic University
Hung Hom, Kowloon, Hong Kong
by.jiang@polyu.edu.hk

Abstract

The rapid proliferation of rumors on social networks poses a significant threat to information integrity. While rumor dissemination forms complex structural patterns, existing detection methods often fail to capture the intricate interplay between textual coherence and propagation dynamics. Current approaches typically represent nodes through isolated textual embeddings, neglecting the semantic flow across the entire propagation path. To bridge this gap, we introduce a novel framework that integrates Large Language Models (LLMs) as a structural augmentation layer for graph-based rumor detection. Moving beyond conventional methods, our framework employs LLMs to evaluate information subchains and strategically introduce a virtual node into the graph. This structural modification converts latent semantic patterns into explicit topological features, effectively capturing the textual coherence that has historically been inaccessible to Graph Neural Networks (GNNs). To ensure reliability, we develop a structured prompt framework that mitigates inherent biases in LLMs while maintaining robust graph learning performance. Furthermore, our proposed framework is model-agnostic, meaning it is not constrained to any specific graph learning algorithm or LLMs. Its plug-and-play nature allows for seamless integration with further fine-tuned LLMs and graph techniques in the future, potentially enhancing predictive performance without the need to modify original algorithms.

1 Introduction

While social media's rapid expansion has revolutionized information sharing, it has simultaneously accelerated the spread of rumors, threatening information credibility and societal stability (Ziari and Charkari, 2025). Detecting rumors in social networks is a critical yet challenging task, as rumors

typically propagate downward along tree-like graph structures over time, with nodes (representing users or posts) and edges (indicating interactions such as replies or retweets) reflecting the flow of information. This temporal nature, combined with the semantic complexity of textual data, requires modeling approaches that can effectively capture structural patterns, temporal evolution, and deep semantic content.

Traditional rumor detection methods, while achieving some success, are constrained by notable limitations. Early machine learning approaches that rely on hand-crafted features struggle to adapt to the diversity and noise inherent in social media data (Zubiaga et al., 2018). Similarly, conventional GNNs, which are primarily designed to model local graph structures, frequently prove inadequate in capturing the subtle rumor propagation pathways and diffusion patterns evident in subchains of varying lengths. LLMs excel at extracting rich semantic features from text, presenting a promising opportunity to address the semantic shortcomings of traditional GNNs (Wang et al., 2024). However, our experiments reveal a key limitation of LLMs when used in isolation for rumor detection: different models exhibit markedly different tendencies due to variations in pre-training data and reinforcement learning objectives. Some LLMs are highly conservative and risk-averse, while others are more aggressive or lenient. Consequently, standalone LLM-based approaches may be inherently unreliable for robust rumor detection, a failure case is shown in Appendix G. Given these conflicting behaviors and the inherent complexity of network data, how to effectively utilize LLMs to enhance the performance of rumor detection remains a critical and open question. To address these challenges, we propose a novel model-agnostic framework that integrates LLMs and GNNs, leveraging structured subchain propagation patterns to enhance rumor detection accuracy.

Our method utilizes LLMs to analyze information flow within tweet subchains, capturing key propagation patterns while leveraging the extensive internal knowledge. While we employ a structured prompt framework to facilitate the processing of network data and mitigate classification biases of LLMs, our primary innovation lies in the architectural enhancement of the rumor graph. We introduce a virtual node representing the rumor status and augment the existing graph structure using nuanced rumor probabilities generated by LLMs for each subchain. This approach is highly efficient as it relies solely on API calls for the LLMs, eliminating the need for local model tuning or intensive storage infrastructure. The enriched graph is subsequently processed to derive robust node representations, thereby facilitating accurate rumor detection through link prediction between the root node and the virtual node. In summary, our study offers two principal contributions: (1) We propose a pioneering, model-agnostic framework that synergistically integrates LLMs and GNNs for rumor detection, enabling effective fusion of semantic and structural information. (2) We develop an innovative approach to capture unexploited propagation features by restructuring the graph with a virtual node and subchain-based connections, enhancing the detection of complex rumor diffusion patterns.

2 Related Work

2.1 Rumor Detection

Rumor detection has evolved from early feature-based machine learning methods to deep learning approaches. Initial techniques used handcrafted features with models like SVM but struggled with social media complexity. Deep learning brought CNNs for spatial features and RNNs/LSTMs for temporal modeling, pioneered by Ma et al. (2016). Transformer models like BERT advanced semantic understanding, while Ma et al. (2017) incorporated propagation structures through kernel learning.

2.2 Large Language Models in Fake News Detection

LLMs have demonstrated significant capabilities in fake news detection, advancing beyond traditional methods. An approach leverages LLMs as judges, utilizing their deep linguistic understanding to evaluate information at scale (Zhou et al., 2023; Li et al., 2024). Another line of research improves LLMs via parameter-efficient tuning, with methods like Cheung and Lam (2023)'s FactLLAMA incorporating external knowledge, while Tian et al. (2025) addressed data quality issues through selective curation. Beyond assessment, LLMs enhance detection systems through data augmentation (Lai et al., 2024) and enable novel frameworks like Ma et al. (2024b)'s approach that models competing perspectives for verification.

2.3 Graph Methods in Rumor Detection

Graph-based methods are vital for rumor detection on social media, capturing information propagation patterns through network structures. Rumors often exhibit unique structural traits, like retweet chains or branching reply threads, naturally represented using graph models. Graph Convolutional Networks (GCNs) aggregate features from connected nodes to create robust embeddings. For instance, Bian et al. (2020) proposed a Bi-Directional GCN to capture bidirectional information flow, enhancing critical node identification. Similarly, Wu et al. (2020) introduced propagation graph neural networks with attention mechanisms to model complex, non-sequential rumor diffusion. Sun et al. (2022) developed a graph adversarial contrastive learning framework, improving robustness via adversarial feature transformations. More recently, Liu et al. (2024) designed a GNN with a bipartite graph to model user correlations alongside tree-structured propagation patterns, integrating social context with diffusion topology. Ma et al. (2024a) proposed a graph sampling and aggregation model (GSMA), enhancing GraphSAGE (Hamilton et al., 2017) with dynamic attention, positional encodings, and sentiment-aware features for better rumor detection, while SePro Zeng et al. (2025) combines LLM reasoning with a GAT-style aggregation of informative social contexts.

3 Problem Statement

Let $\mathcal{N} = \{N_1, N_2, \ldots, N_s\}$ denote the set of source news articles, where each $N_i \in \mathcal{N}$ represents an individual news item. Associated with each N_i is a set of reactions $\mathcal{R}_i = \{R_{i1}, R_{i2}, \ldots, R_{it_i}\}$, forming a multi-level discussion structure beneath each original news post. Specifically, 1) each news post contains a large number of user replies; 2) these replies may directly respond to the news post or be nested under existing replies as sub-replies, thus forming a tree-like dialogue structure. This tree propagation structure corresponds exactly to the way rumors spread. As shown in Figure 1(a), the yellow root node at the top represents the source news, and the child nodes below are all reaction posts.

In rumor detection tasks, the problem is formulated as a binary classification task. The objective is to train a model using news instances labeled with ground-truth values $y \in \{\text{Rumor}, \text{Non-Rumor}\}$, enabling accurate prediction of labels for unseen test news items. Our work focuses on developing a model-agnostic framework for rumor detection, designed to seamlessly integrate with various graph learning algorithms and LLMs.

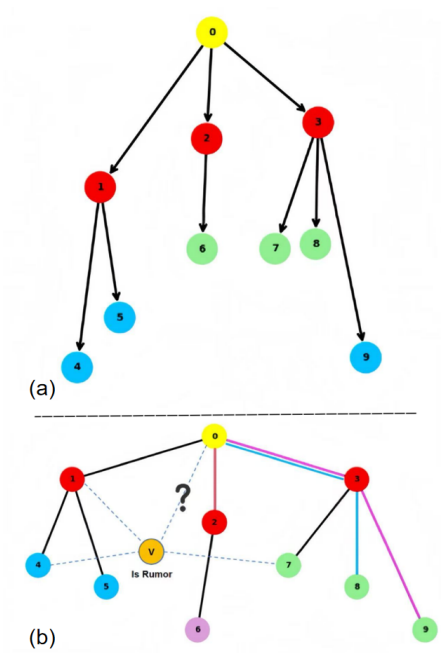

Figure 1: Rumor propagation patterns in social media.

4 Algorithm

For each source news, a directed graph $\mathbb{G} = (\mathbb{V}, \mathbb{E})$ is constructed: $\mathbb{V} = \{v_0, v_1, \ldots, v_n\}$ is the set of nodes, where v_0 is the source news (root node) and each child node $v_i \in \mathbb{V} \setminus \{v_0\}$ represents a reply post. $\mathbb{E}$ is the set of edges and the direction of the edge is the same as the direction of information propagation. The text associated with node v_i is denoted as $\text{text}(v_i)$. Each node v_i is assigned a feature vector x_i, extracted using BERT from the final-layer [CLS] representation as follows:

$$x_i = \text{BERT}(\text{text}(v_i)).$$

Subchain Construction: For each child node $v_i \in \mathbb{V} \setminus \{v_0\}$, there exists a unique path (subchain) from the root node v_0 to v_i, denoted as:

$$\text{path}(v_0, v_i) = (v_0, v_{p_1}, v_{p_2}, \ldots, v_{p_k}, v_i),$$

where v_{p_j} are intermediate nodes on the path, and $k \geq 0$. The text information of the subchain is concatenated using a separator token [SEP]:

$$\begin{aligned}\text{info}(\text{path}(v_0, v_i)) = \text{text}(v_0)\ [\text{SEP}]\ \text{text}(v_{p_1}) \cdots \\ \cdots\ [\text{SEP}]\ \text{text}(v_i).\end{aligned}$$

This chain-like information flow captures rich contextual information from the root to each child node. By leveraging this structure, our approach aims to enhance the capability of LLMs and graph learning algorithms to detect rumors effectively. As depicted in Figure 1(b), the red path from the root node to child node 2 represents the subchain associated with child node 2. By analogy, the entire graph contains as many subchains as there are child nodes, with each subchain corresponding to the unique path from the root node to a child node.

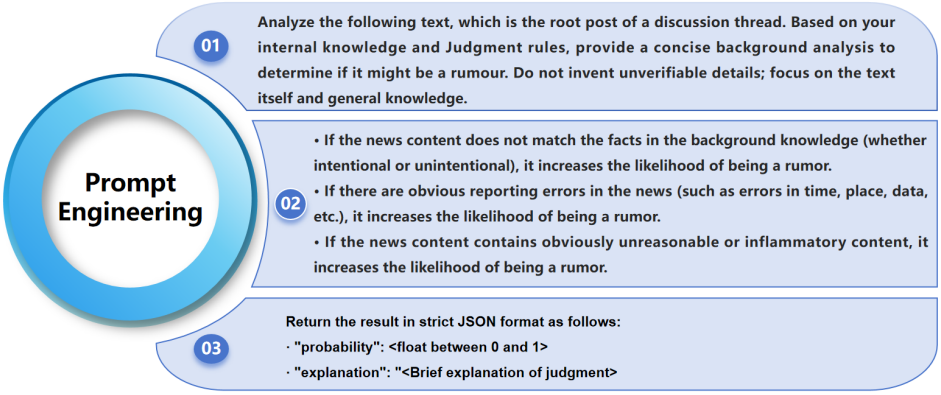

Figure 2: An overview of our prompt design methodology.

Virtual Node and Edge Augmentation in Graph Structures: To leverage the capabilities of LLMs and the graphical structure of information flows, we introduced a virtual node v^*, labeled "is Rumor", which initially lacks feature information. Initial feature of v^* is set to the zero vector and its neighbors do not aggregate messages from v^*. This means that even in a bidirectional GNN, a virtual node only has edges pointing to it. We then use a LLM to process each subchain and outputs the probability that the source news is a rumor. For each child node v_i, the LLM takes the concatenated text information of the subchain and outputs a probability:

$$P_{\text{rumor}}(v_i) = \text{LLM}(\text{info}(\text{path}(v_0, v_i))), \tag{1}$$

where $P_{\text{rumor}}(v_i) \in [0, 1]$ represents the likelihood that the source news is a rumor based on the subchain from the root node to node v_i. For each child node v_i, if the LLM probability exceeds a predefined threshold θ:

$$P_{\text{rumor}}(v_i) > \theta,$$

a directed edge is established from v_i and v^*, denoted as $(v_i, v^*) \in \mathbb{E}'$, where $\mathbb{E}'$ is the set of new edges added to the graph. The updated graph is denoted as $\mathbb{G}' = (\mathbb{V} \cup \{v^*\}, \mathbb{E} \cup \mathbb{E}')$. As shown in Figure 1(b), assume that the LLM assigns probabilities to the subchains of child nodes 1, 4, and 7 exceeding the threshold, edges are established between these nodes and the virtual node. Through these operations, we enhance the graph structure by introducing a virtual node and leveraging LLMs' reasoning capabilities to connection it to rumor propagation across the subchains. The rumor detection problem then reduces to predicting whether there is a link between the root node v_0 and the virtual node v^*. Next we provide further details on how to retrieve the probability (1) from a LLM. It has been shown that carefully designed prompt engineering can enhance the ability of LLMs to identify rumors (Yan et al., 2024; Shehata, 2024). In our paper, we propose a specialized prompt engineering framework. As depicted in Step 1 of Figure 2, the input prompts guide the LLM to leverage its internal knowledge and predefined rules to briefly analyze the root post's background and evaluate its potential as a rumor. In order to solve the problem that LLMs have different evaluation criteria for the same news (Huang, 2025; Mohanty, 2025), we prompt the model to query and compile detailed background information on the event when processing the root node's original news post, storing it in a persistent knowledge base. In Step 2 of Figure 2, subsequent subchain evaluations strictly adhere to this knowledge base alongside the prompt's criteria, ensuring consistent and reliable assessments throughout the process. As will be seen in Section 5.3, this prompting strategy successfully enhances the reliability of LLMs for rumor detection. Figure 3

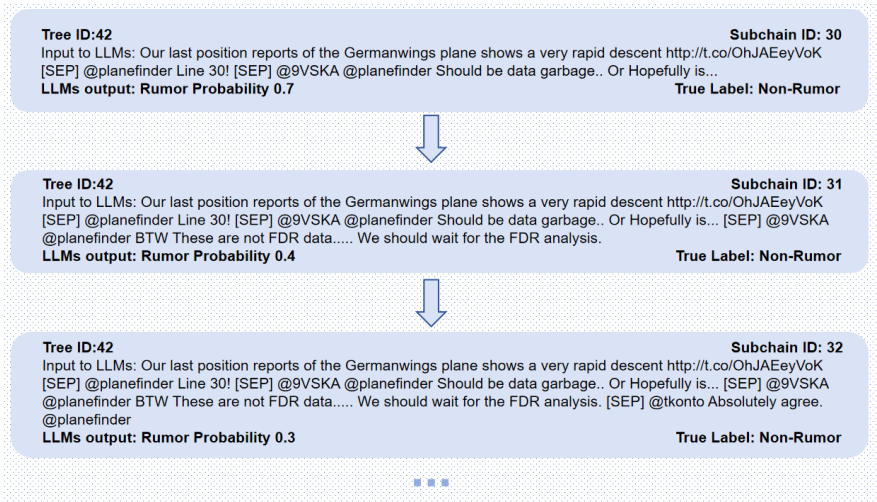

Figure 3: LLM processes information flow.

presents an example illustrating how the LLM evaluates rumor probabilities across subchains in a propagation tree, using breaking news coverage of the Airbus A320 Germanwings crash. The true label of the root news is "Non-rumor". The subchains with IDs 30, 31, and 32 differ by only one child node. We can see a clear downward trend in probability as the number of child nodes in the subchain increases. This sequential evaluation demonstrates the model's capability to process evolving information threads, dynamically adjusting probabilities based on accumulated context while maintaining consistency through the established knowledge base. The complete prompt templates are provided in Appendix H for full reproducibility.

To address extreme scenarios where the LLM-augmented graph contains notably few virtual edges, which may result from limitations in the model's knowledge base or reasoning capabilities,we introduce a mitigation strategy. This approach aims to diminish the undue influence of the LLM in such outliers while leveraging graph learning methods to correct biases (Li et al., 2025; Hang, 2025). Specifically, if the number of virtual edges falls below a predefined minimum threshold (indicating sparse connections), we retain connections to some child nodes ranked top by the rumor probability assigned by LLM, ensuring a baseline level of virtual edges for graph propagation to refine and rectify potential LLM misjudgments. Formally, for child nodes $\{v_1, \ldots, v_n\}$ with corresponding rumor probabilities $P = \{p_1, \ldots, p_n\}$, the original method adds a virtual edge from the virtual node to v_i if $p_i > \theta$. However, if the resulting edge count $|E_v| < \lceil \gamma n \rceil$, we sort P in descending order and connect the virtual node to the top $k = \lceil \gamma n \rceil$ child nodes, even if $p_i \leq \theta$. This can be expressed as:

$$E_v = \begin{cases} \{(v_i, v^*) \mid p_i > \theta\} & \text{if } |E_v| \geq \lceil \gamma n \rceil \\ \{(v_{\sigma(j)}, v^*) \mid 1 \leq j \leq k\} & \text{otherwise.} \end{cases}$$

Here, v^* denotes the virtual node, σ is the permutation sorting indices by $p_{\sigma(1)} \geq \cdots \geq p_{\sigma(n)}$, and $k = \lceil \gamma n \rceil$ ensures at least a proportional subset of edges. In Appendix D, we provide an ablation study showing that $\gamma = 0.2$ yields consistently stable performance, and that the root-virtual link prediction outperforms root-only classification under the same backbone. Appendix E further analyzes virtual-node connectivity and depth effects, while Appendix F reports subchain token statistics to characterize practical input sizes. Both provide further support for the robustness and promising performance of our method.

To demonstrate the enhancing effect of our framework on GNN, we will use Bidirectional Graph

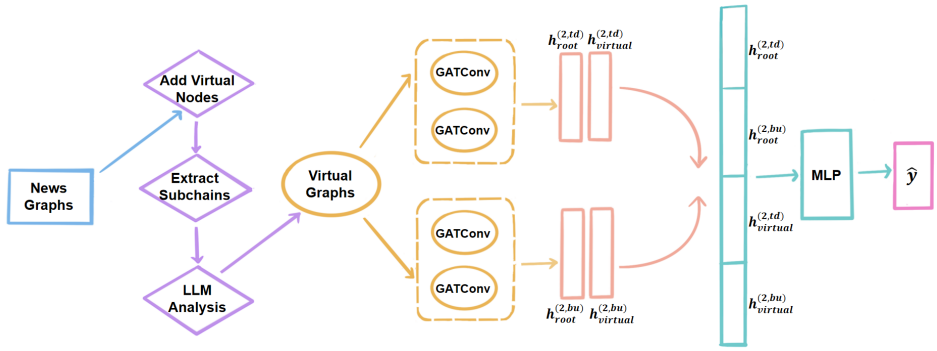

Figure 4: LLM-enhanced Bi-GAT model.

Attention Network (Bi-GAT) as an example. To implement Bi-GAT with the virtual node, the given graph $\mathbb{G} = (\mathbb{V}, \mathbb{E})$ is extended to $\tilde{\mathbb{G}} = (\tilde{\mathbb{V}}, \tilde{\mathbb{E}})$, with $\tilde{\mathbb{V}} = \mathbb{V} \cup \{v^*\}$ and $\tilde{\mathbb{E}} = \{(v_i, v_j), (v_j, v_i) : (v_i, v_j) \in \mathbb{E}\} \cup \mathbb{E}'$.

Graph Attention Convolution (GATConv) The GATConv function implements the graph attention mechanism proposed by Veličković et al. (2018), which computes node representations by attending over neighboring nodes with learned attention coefficients. For a graph $\tilde{\mathbb{G}} = (\tilde{\mathbb{V}}, \tilde{\mathbb{E}})$ with node features $\mathbf{x}_i \in \mathbb{R}^F$ for node $v_i \in \tilde{\mathbb{V}}$ (node feature is set to be zero for v^*), the `GATConv` layer computes attention scores for its neighbors $v_j \in \mathcal{N}(i)$, where $\mathcal{N}(i) = \{v_j \mid (v_i, v_j) \in \tilde{\mathbb{E}}\}$ is the set of neighboring nodes. The attention score e_{ij} between nodes v_i and v_j is calculated as:

$$e_{ij} = \text{LeakyReLU}\left(\mathbf{a}^T[\mathbf{W}_q\mathbf{x}_i \parallel \mathbf{W}_k\mathbf{x}_j]\right),$$

where $\mathbf{W}_q$, $\mathbf{W}_k$, $\mathbf{W}_v \in \mathbb{R}^{F' \times F}$ are weight matrices that transform the input features to the hidden dimension. $\mathbf{a} \in \mathbb{R}^{2F'}$ is the attention parameter vector. $[\mathbf{W}_q\mathbf{x}_i \parallel \mathbf{W}_k\mathbf{x}_j] \in \mathbb{R}^{2F'}$ is the concatenation of the transformed features of nodes v_i and v_j. Then the attention coefficients α_{ij} are normalized across neighbors using the softmax function and use the attention coefficient to weight the neighbor value vector to update the feature of node i:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad \mathbf{h}_i = \sum_j \alpha_{ij}\mathbf{W}_v x_j,$$

where $k, j \in \mathcal{N}(i)$. To stabilize and enhance the attention mechanism, multiple attention heads are employed. Each head $h = 1, \ldots, H$ has its own weight matrix $\mathbf{W}_v^{(h)} \in \mathbb{R}^{F' \times F}$ and attention vector $\mathbf{a}^{(h)} \in \mathbb{R}^{2F'}$. The output of each head is:

$$\mathbf{h}_i^{(h)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(h)}\mathbf{W}_v^{(h)}\mathbf{x}_j,$$
$$\mathbf{h}_i = \|_{h=1}^{H} \mathbf{h}_i^{(h)} \in \mathbb{R}^{HF'},$$

where $\alpha_{ij}^{(h)}$ is the attention coefficient for head h. $\mathbf{h}_i \in \mathbb{R}^{HF'}$ is the final output for node v_i concatenating the head outputs.

Top-Down GAT (TD-GAT) Simulate the information transmission from the “high-level" nodes to the “low-level" nodes of the graph, which is suitable for capturing causal relationships.

- **First GATConv Layer**:

$$\mathbf{h}_i^{(1,\text{td})} = \text{ReLU}\left(\|_{h=1}^{H} \sum_{j \in \mathcal{N}_{out}(i)} \alpha_{ij}^{(h)} \mathbf{W}_{\text{td1}}^{(h)} \mathbf{x}_j \right),$$

where $\mathcal{N}_{out}(i)$ is the out-neighbors of node i and $\mathbf{h}_i^{(1,\text{td})} \in \mathbb{R}^{HF'}$.

- **Second GATConv Layer**:

$$\mathbf{h}_i^{(2,\text{td})} = \frac{1}{H} \sum_{h=1}^{H} \sum_{j \in \mathcal{N}_{out}(i)} \alpha_{ij} \mathbf{W}_{\text{td2}} [\mathbf{h}_j^{(1,\text{td})} \parallel \mathbf{x}_\text{r}],$$

where x_r is the feature of root node and $\mathbf{h}_i^{(2,\text{td})} \in \mathbb{R}^{F'}$.

Bottom-Up GAT (BU-GAT) Simulate the aggregation of features from low-level nodes to high-level nodes of the graph.

- **First GATConv Layer**:

$$\mathbf{h}_i^{(1,\text{bu})} = \text{ReLU}\left(\|_{h=1}^{H} \sum_{j \in \mathcal{N}_{in}(i)} \alpha_{ij}^{(h)} \mathbf{W}_{\text{bu1}}^{(h)} \mathbf{x}_j \right),$$

where $\mathcal{N}_{in}(i)$ is the in-neighbors of node i.

- **Second GATConv Layer**:

$$\mathbf{h}_i^{(2,\text{bu})} = \frac{1}{H} \sum_{h=1}^{H} \sum_{j \in \mathcal{N}_{in}(i)} \alpha_{ij} \mathbf{W}_{\text{bu2}} [\mathbf{h}_j^{(1,\text{bu})} \parallel \mathbf{x}_\text{r}].$$

Feature Fusion and Edge Classification

- **Feature Extraction**: Extract $\mathbf{h}_{\text{root}}^{(2,\text{td})}$, $\mathbf{h}_{\text{virtual}}^{(2,\text{td})}$, $\mathbf{h}_{\text{root}}^{(2,\text{bu})}$, and $\mathbf{h}_{\text{virtual}}^{(2,\text{bu})}$.
- **Fusion**: Concatenate into $\mathbf{c} = [\mathbf{h}_{\text{root}}^{(2,\text{td})}, \mathbf{h}_{\text{root}}^{(2,\text{bu})}, \mathbf{h}_{\text{virtual}}^{(2,\text{td})}, \mathbf{h}_{\text{virtual}}^{(2,\text{bu})}] \in \mathbb{R}^{4F'}$.
- **Classification**: $\mathbf{z} = \mathbf{W}_{\text{edge}}\mathbf{c} + \mathbf{b}_{\text{edge}}$, $\hat{y}_{\text{edge}} = \sigma(\mathbf{z})$, where $\mathbf{W}_{\text{edge}} \in \mathbb{R}^{4F'}$, $\mathbf{b}_{\text{edge}} \in \mathbb{R}$, and σ is the sigmoid function.

Model and Training We use the binary cross-entropy function. For a batch size of N, the loss function is defined as

$$\begin{aligned} \textbf{Loss} = -\frac{1}{N} \sum_{i=1}^{N} \omega_i [& y_i \log(\sigma(z_i)) \\ & + (1 - y_i) \log(1 - \sigma(z_i))] \end{aligned}$$

where z_i is the positive class logit output by the model (i.e. the logit score of the virtual node and the root node with an edge). y_i is the true label of the source news, and ω_i is the weight determined by y_i's category and pos_weight. pos_weight is set to the negative-to-positive class ratio, which increases the loss weight for positive (minority) samples to prioritize their correct prediction and counteract class imbalance. Parameters are trained using the Adam optimizer with backpropagation to optimize all components of the Bi-GAT model. We use a 7:1:2 train/validation/test split and select θ only on validation data. For PHEME, we apply overall stratified sampling on the full dataset (not per event), then pool validation instances from all five events to choose one shared threshold via Youden's (J). For Weibo, which has no event partition, we similarly use a global split and one validation-selected threshold. The hyperparameters were set as follows: learning rate = 0.00005, weight decay = 1e-3, dropout rate = 0.3, and a maximum of 150 training epochs. An early stopping mechanism was used; during training, the F1 score on the validation set is continuously monitored, and training is automatically stopped if the score failed to exceed the historical best value for 20 consecutive epochs, thus preventing model overfitting.

5 Experiments

5.1 Datasets and LLMs

We experiment on five news events from the PHEME dataset: Charlie Hebdo shooting (Charlie Hebdo); Killing of Michael Brown (Ferguson); Germanwings Flight 9525 (Germanwings crash); 2014 shootings at Ottawa (Ottawa Shooting); Lindt Cafe siege (Sydney Siege); and Weibo dataset (Ma et al., 2016), as shown in Table 1. After preprocessing, we removed graphs without replies. The same preprocessing was applied to all baselines for fair comparison. This experiment uses the DeepSeek-V3 as the base model. Additional numerical results using Qwen-Plus are reported in Appendix B & C. Both PHEME and Weibo are public datasets.

News Event	Non-Rumor	Rumor	Total
Charlie Hebdo	1621	458	2079
Ferguson	859	284	1143
Germanwings Crash	231	238	469
Ottawa Shooting	420	470	890
Sydney Siege	699	522	1221
Weibo	2313	2351	4664

Table 1: Statistics of PHEME and Weibo Datasets.

5.2 Evaluation metrics

Table 2 presents the fundamental metrics for assessing classification model performance: Accuracy, Precision, Recall, and F1 Score.

We name our proposed framework LLM-VN and compare it with existing methods as comparative baselines: Bi-GCN (Bian et al., 2020) captures bidirectional dependencies in trees; RvNN (Socher et al., 2011) recursively composes child nodes into parents; GAT (Veličković et al., 2018) uses attention to weigh neighbor importance; HD-TRANs (Ma and Gao, 2020) integrates Transformer with graph networks for dynamic graphs; LINE (Tang et al., 2015) provides large-scale static network embeddings; DDGCN (Korban and Li, 2020) models spatiotemporal features in dynamic directed graphs; GraphSAGE (Hamilton et al., 2017) samples and aggregates local features inductively;

Metric	Formula	Term
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	TP
Precision	$\frac{TP}{TP + FP}$	TN
Recall	$\frac{TP}{TP + FN}$	FP
F1 Score	$2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$	FN

Table 2: Classification Metrics, Terms, and Formulas.

GSMA (Ma et al., 2024a) enhances GraphSAGE with attention and position encoding for rumor detection. SePro(Zeng et al., 2025) refines social contexts via semantic-propagation collaboration and Chain-of-Clue prompting, improving LLMs' performance.

The 'LLM-VN' enhanced version of the above baselines are denoted as '**LLM-VN+Baseline**'. We also compare our approach to simple model average where the final probability of rumors is a weighted average of the probability from LLM with well-designed prompt and the probability from the baseline method. The optimal weight is tuned via line search. We denote this as '**LLM+Baseline**'.

5.3 Results and analysis

As shown in Table 3, on the PHEME dataset, LLMs perform poorly in rumor identification without carefully designed prompts. Acc. (R) measures the proportion of correctly identified "rumor" instances among all true rumors, while Acc. (N) does the same for "non-rumor" instances.

News Event	Acc. (R)	Acc. (N)
Charlie Hebdo	15.54%	83.20%
Ferguson	41.55%	89.76%
Germanwings crash	23.11%	91.77%
Ottawa shooting	16.81%	87.62%
Sydney Siege	33.14%	78.25%

Table 3: Performance of LLMs under basic prompts.

The performance of our proposed framework 'LLM-VN' combined with Bi-GAT on the PHEME dataset are presented in Table 4:

News Event	**Acc.**	**Pre.**	**Rec.**	**F1**	**AUC**
Charlie Hebdo	0.923	0.943	0.967	0.955	0.962
Ferguson	0.860	0.818	0.892	0.853	0.903
Germanwings Crash	0.894	0.914	0.961	0.937	0.827
Ottawa Shooting	0.876	0.883	0.987	0.932	0.846
Sydney Siege	0.829	0.866	0.905	0.885	0.858

Table 4: Performance of LLM-VN+Bi-GAT.

From Table 4, the model achieves strong performance on most news events, with accuracy often exceeding 85%, along with high precision, recall, and F1 scores. Our results on PHEME suggest that the framework remains effective even when reply evidence is relatively sparse. To better illustrate the model-agnostic nature of our approach and demonstrate the added benefits of the virtual node beyond prompt engineering, we present in Table 5 a comprehensive comparison of graph learning methods for rumor detection on the Weibo and PHEME datasets. It evaluates baseline models in three setups: the original baseline, LLM+Baseline, and LLM-VN+Baseline. To ensure a fair comparison,

	PHEME Dataset					Weibo Dataset				
Method	**Acc**	**Prec**	**Rec**	**F1**	**Class**	**Acc**	**Prec**	**Rec**	**F1**	**Class**
Bi-GCN	0.824	0.753/0.861	0.734/0.872	0.741/0.865	R/N	0.963	0.948/0.970	0.946/0.972	0.947/0.971	R/N
LLM+Bi-GCN	0.830	0.758/0.862	0.744/0.850	0.750/0.855	R/N	0.971	0.962/0.975	0.959/0.979	0.960/0.976	R/N
LLM-VN+Bi-GCN	**0.842**	0.772/0.872	0.752/0.890	0.761/0.884	R/N	**0.988**	0.976/0.992	0.978/0.991	0.976/0.992	R/N
RvNN	0.763	0.689/0.796	0.587/0.858	0.631/0.825	R/N	0.771	0.723/0.782	0.681/0.795	0.646/0.817	R/N
LLM+RvNN	0.783	0.777/0.811	0.754/0.802	0.761/0.808	R/N	0.775	0.692/0.793	0.595/0.800	0.638/0.796	R/N
LLM-VN+RvNN	**0.807**	0.773/0.835	0.696/0.896	0.784/0.842	R/N	**0.781**	0.773/0.791	0.796/0.766	0.784/0.778	R/N
Graphsage	0.842	0.772/0.876	0.820/0.878	0.795/0.877	R/N	0.963	0.956/0.972	0.953/0.975	0.954/0.973	R/N
GSMA	0.848	0.834/0.856	0.823/0.851	0.840/0.860	R/N	0.974	0.967/0.973	0.953/0.982	0.960/0.977	R/N
LLM-VN+Graphsage	**0.866**	0.822/0.887	0.825/0.879	0.820/0.892	R/N	**0.981**	0.978/0.972	0.933/0.992	0.955/0.982	R/N
GAT	0.811	0.733/0.877	0.541/0.798	0.405/0.833	R/N	0.947	0.939/0.961	0.936/0.943	0.938/0.962	R/N
LLM+GAT	0.823	0.799/0.826	0.796/0.820	0.804/0.828	R/N	0.960	0.944/0.969	0.939/0.953	0.941/0.961	R/N
SePro	0.831	0.778/0.852	0.762/0.864	0.770/0.858	R/N	0.950	0.947/0.954	0.948/0.953	0.947/0.953	R/N
LLM-VN+GAT	**0.847**	0.823/0.868	0.860/0.850	0.800/0.870	R/N	**0.982**	0.977/0.987	0.972/0.985	0.981/0.989	R/N
HD-TRANs	0.766	0.656/0.783	0.697/0.755	0.676/0.768	R/N	0.974	0.957/0.979	0.946/0.978	0.952/0.979	R/N
LLM+HD-TRANs	0.779	0.767/0.784	0.766/0.782	0.780/0.777	R/N	0.980	0.961/0.984	0.952/0.988	0.959/0.987	R/N
LLM-VN+HD-TRANs	**0.796**	0.696/0.811	0.737/0.792	0.716/0.802	R/N	**0.991**	0.963/0.981	0.960/0.984	0.962/0.989	R/N
LINE	0.744	0.732/0.749	0.730/0.750	0.733/0.749	R/N	0.790	0.763/0.802	0.771/0.811	0.760/0.796	R/N
LLM+LINE	0.759	0.753/0.760	0.750/0.758	0.755/0.763	R/N	0.803	0.768/0.804	0.775/0.806	0.763/0.803	R/N
LLM-VN+LINE	**0.786**	0.747/0.794	0.745/0.782	0.746/0.797	R/N	**0.811**	0.802/0.820	0.804/0.816	0.800/0.823	R/N
DDGCN	0.855	0.877/0.831	0.763/0.892	0.816/0.860	R/N	0.948	0.941/0.965	0.933/0.970	0.937/0.967	R/N
LLM+DDGCN	0.860	0.855/0.866	0.852/0.861	0.856/0.867	R/N	0.954	0.950/0.959	0.949/0.955	0.952/0.963	R/N
LLM-VN+DDGCN	**0.876**	0.858/0.861	0.832/0.867	0.845/0.864	R/N	**0.984**	0.979/0.988	0.978/0.985	0.982/0.990	R/N

Table 5: Performance Comparison on PHEME and Weibo Datasets.

our proposed prompting strategy (Appendix H) is applied to all evaluated baselines. We follow the standard benchmark setting on PHEME and Weibo, rather than a strictly time-grounded early detection protocol. In our framework, pretrained LLM knowledge serves only as an auxiliary signal for enhancing GNN-based rumor detection, while the final prediction is still made by the downstream graph model. From table 5, we can see that the LLM-VN-enhanced methods demonstrate significant advantages on both the PHEME and Weibo datasets. Compared to the original baseline method and the model average approaches (LLM+Baseline), our framework shows substantial improvements in key metrics such as accuracy, precision, recall, and F1 score. Especially on the PHEME dataset, methods like LLM-VN+DDGCN exhibit particularly outstanding performance in accuracy and F1 score. On the Weibo dataset, methods such as LLM-VN+Bi-GCN and LLM-VN+GAT also outperform the baseline, particularly in terms of precision and recall for the non-rumor category, demonstrating higher classification stability. In contrast, the improvement of LLM+Baseline is less robust than its VN-enhanced version, especially in cases of complex propagation structures or imbalanced data distributions. On the Weibo dataset, methods outperform PHEME due to Weibo's larger, deeper, and richer propagation trees. Overall, the LLM-VN framework enhances generalization and classification accuracy by integrating LLMs' semantic understanding with graph models' structured processing, particularly on complex social media propagation patterns.

6 Conclusions

We propose a general framework to enhance the performance of graph learning methods for rumor prediction by leveraging LLMs. By employing LLMs to analyze subchains and assign rumor probabilities, we augment the graph with a virtual "is Rumor" node. Intuitively, when LLMs predict

high rumor probabilities across many subchains, the virtual node shares more neighbors with the root node, leading to convergent embeddings during GNN propagation. This enhances the likelihood of link prediction between the root and virtual nodes, effectively classifying the source news as a rumor. A key strength of our design is its robustness to imperfect LLM signals: the mitigation mechanism reduces overreliance on noisy or weak evidence, such as in early-stage diffusion or sparse and biased replies. Notably, our method is a modular framework that leverages LLMs to retrieve and reason over subchain evidence, injecting this signal into a propagation graph for GNN-based prediction. The model-agnostic design allows it to be paired with different GNN backbones and LLMs. As GNNs and LLMs continue to advance, our framework will inherently improve alongside them. Furthermore, as illustrated in Appendix A, the integration of LLM in our approach relies solely on API calls, ensuring low computation cost. The processing time per news is generally around 10 seconds for the PHEME dataset (with less than 100 nodes), and 30-60 seconds for the Weibo dataset (with thousands of nodes).

Limitations

A limitation of our approach emerges when handling extremely large-scale graphs. Despite the cost and runtime analysis in Appendix A, the reliance on LLM APIs can still introduce non-negligible latency, preventing fully real-time responses for very large graphs.

References

Tian Bian, Xiao Xi, Tingyang Xu, Peilin Zhao, Wenbing Huang, Yu Rong and Junzhou Huang. 2020. Rumor Detection on Social Media with Bi-Directional Graph Convolutional Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 34, No. 1, pp. 549-556). AAAI Press. doi:10.1609/aaai.v34i01.5393.

Tsun Hin Cheung and Kin Man Lam. 2023. FactLLaMA: Optimizing Instruction-Following Language Models with External Knowledge for Automated Fact-Checking. In *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)* (pp. 846-853). IEEE. doi:10.1109/APSIPAASC58517.2023.10317251.

William L. Hamilton, Rex Ying and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30* (Vol. 30, pp. 1024-1034). Curran Associates.

Ching Nam Hang, Peiduo Yua and Chee Wei Tan. 2025. TrumorGPT: Graph-Based Retrieval-Augmented Large Language Model for Fact-Checking. *arXiv:2505.07891*.

Tianyi Huang, Jingyuan Yi, Peiyang Yu and Xiaochuan Xu. 2025. Unmasking Digital Falsehoods: A Comparative Analysis of LLM-Based Misinformation Detection Strategies. *arXiv:2503.00724*.

Matthew Korban and Xin Li. 2020. DDGCN: A Dynamic Directed Graph Convolutional Network for Action Recognition. In *European Conference on Computer Vision* (pp. 761-776). Springer. doi:10.1007/978-3-030-58565-5_45.

Sumeet Kumar and Kathleen Carley. 2019. Tree LSTMs with Convolution Units to Predict Stance and Rumor Veracity in Social Media Conversations. In *Proceedings of the 57th Annual Meeting*

of the Association for Computational Linguistics (pp. 5047-5058). Association for Computational Linguistics. doi:10.18653/v1/P19-1498.

Jianqiao Lai, Xinran Yang, Wenyue Luo, Linjiang Zhou, Langchen Li, Yongqi Wang and Xiaochuan Shi. 2024. RumorLLM: A Rumor Large Language Model-Based Fake-News-Detection Data-Augmentation Approach. *Applied Sciences*, 14(8):3532. doi:10.3390/app14083532.

Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Frameworks. *arXiv:2412.05579*

Guoyi Li, Die Hu, Zongzhen Liu, Xiaodan Zhang, Honglei Lyu. 2025. Semantic Reshuffling with LLM and Heterogeneous Graph Auto-Encoder for Enhanced Rumor Detection. In *Proceedings of the 31st International Conference on Computational Linguistics* (pp. 8557-8572). Association for Computational Linguistics. doi:10.18653/v1/2025.coling-main.572.

Tianrui Liu, Qi Cai, Changxin Xu, Bo Hong, Fanghao Ni, Yuxin Qiao and Tsungwei Yang. 2024. Rumor Detection with A Novel Graph Neural Network Approach. *Academic Journal of Science and Technology*, 10(1):1-6. doi:10.54097/farmdr42.

Jing Ma, Wei Gao, Prasenjit Mitra, Sejeong Kwon, Bernard J. Jansen, Kam-Fai Wong and Meeyoung Cha. 2016. Detecting Rumors from Microblogs with Recurrent Neural Networks. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI 2016)* (pp. 3818-3824). AAAI Press.

Jing Ma, Wei Gao and Kam-Fai Wong. 2017. Detect Rumors in Microblog Posts Using Propagation Structure via Kernel Learning. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 708-717). Association for Computational Linguistics. doi:10.18653/v1/P17-1066.

Jing Ma and Wei Gao. 2020. Debunking Rumors on Twitter with Tree Transformer. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING)* (pp. 5455-5466). Association for Computational Linguistics. doi:10.18653/v1/2020.coling-main.476.

Manfu Ma, Cong Zhang, Yong Li, Jiahao Chen and Xuegang Wang. 2024. Rumor Detection Model with Weighted GraphSAGE Focusing on Node Location. *Scientific Reports*, 14(1):27127. doi:10.1038/s41598-024-76738-7.

Xiaoxiao Ma, Yuchen Zhang, Kaize Ding, Jian Yang, Jia Wu and Hao Fan. 2024. On Fake News Detection with LLM Enhanced Semantics Mining. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 508-521). Association for Computational Linguistics. doi:10.18653/v1/2024.emnlp-main.31.

Suvendu Mohanty. 2025. Fine-Grained Bias Detection in LLM: Enhancing Detection Mechanisms for Nuanced Biases. *arXiv:2503.06054*.

Dahlia Shehata, Robin Cohen and Charles Clarke. 2024. Rumour Evaluation with Very Large Language Models. *arXiv:2404.16859*.

Richard Socher, Cliff Chiung-Yu Lin, Andrew Y. Ng and Christopher D. Manning. 2011. Parsing Natural Scenes and Natural Language with Recursive Neural Networks. In *Proceedings of the 28th International Conference on Machine Learning* (pp. 129-136). Omnipress.

Tiening Sun, Zhong Qian, Sujun Dong, Peifeng Li and Qiaoming Zhu. 2022. Rumor Detection on Social Media with Graph Adversarial Contrastive Learning. In *Proceedings of the ACM Web Conference 2022* (pp. 2789-2797). ACM. doi:10.1145/3485447.3511999.

Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan and Qiaozhu Mei. 2015. LINE: Large-scale Information Network Embedding. In *Proceedings of the 24th International Conference on World Wide Web (WWW)* (pp. 1067-1077). ACM. doi:10.1145/2736277.2741093.

Zhiliang Tian, Jingyuan Huang, Zejiang He, Zhen Huang, Menglong Lu, Linbo Qiao, Songzhu Mei, Yijie Wang and Dongsheng Li. 2025. LLM-based Rumor Detection via Influence Guided Sample Selection and Game-based Perspective Analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (pp. 1378-1391). Association for Computational Linguistics. doi:10.18653/v1/2025.acl-long.1378.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations* . OpenReview.net.

Bo Wang, Jing Ma, Hongzhan Lin, Zhiwei Yang, Ruichao Yang, Yuan Tian and Yi Chang. 2024. Explainable Fake News Detection With Large Language Model via Defense Among Competing Wisdom. In *Proceedings of the ACM Web Conference 2024* (pp. 2452-2463). ACM. doi:10.1145/3589334.3645471.

Zhiyuan Wu, Dechang Pi, Junfu Chen, Meng Xie and Jianjun Cao . 2020. Rumor Detection Based on Propagation Graph Neural Network with Attention Mechanism. *Expert Systems with Applications*, 158:113595. doi:10.1016/j.eswa.2020.113595. doi:10.1016/j.eswa.2020.113595.

Yeqing Yan, Peng Zheng and Yongjun Wang. 2024. Enhancing Large Language Model Capabilities for Rumor Detection with Knowledge-Powered Prompting. *Engineering Applications of Artificial Intelligence*, 133:108259. doi:10.1016/j.engappai.2024.108259.

Yirong Zeng, Xiao Ding, Bibo Cai, Ting Liu, and Bing Qin. 2025. Moderation Matters: Exploring Large Language Models for Effective Rumor Detection on Social Media. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 2537–2552). Association for Computational Linguistics.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer and Omer Levy. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems 36*. Curran Associates.

Mohammad Ziari and Nasrollah Moghaddam Charkari. 2025. Rumor Detection and Propagation on Social Networks: A Survey. *Expert Systems with Applications*, 263:128798. doi:10.1016/j.eswa.2025.128798.

Arkaitz Zubiaga, Ahmet Aker, Kalina Bontcheva, Maria Liakata and Rob Procter. 2018. Detection and Resolution of Rumours in Social Media: A Survey. *ACM Computing Surveys*, 51(2):1-36. doi:10.1145/3161603.

A Computational Cost

On a single thread, both Qwen-Plus and DeepSeek-V3 process more than 1,200 nodes per hour, and this can be further accelerated with multi-threading. Since responses are limited to probabilities only, token usage and costs remain low.

On a standard workstation (≥16 cores, ≥32 GB RAM, stable ≥100 Mbps network) using simple asynchronous multi-threading (100 parallel API calls to DeepSeek-V3 or Qwen-Plus), we measured the actual computation time and API costs for processing the two datasets. As shown in Table 6, these results demonstrate the practical deployability of our framework in resource-constrained environments. Graphs in the PHEME dataset typically have only a few dozen nodes and take about 10 seconds to process each news item, while those in the Weibo dataset usually contain thousands of nodes and take 30–60 seconds per items.

Metric	PHEME	Weibo
Trees	5,447	3,805
Posts	96,344	3,804,357
Time (100 threads)	40–50 min	35–40 h
API Cost (USD)	5–10	80–100

Table 6: Computational cost of the LLMs on a standard workstation.

B LLMs Bias

To further demonstrate the biases inherent in LLMs, we evaluated their performance on the PHEME dataset using Qwen-Plus. In Table 7, our analysis reveals that Qwen exhibits a markedly different bias compared to DeepSeek. This divergence primarily stems from their distinct training data and reinforcement learning strategies. Specifically, Qwen demonstrates a more subjective and aggressive stance, it tends to err on the side of caution by prioritizing the avoidance of false negatives, even at the cost of increasing false positives. In contrast, DeepSeek adopts a considerably more conservative and neutral approach, leaning towards minimizing false positives rather than risking false negatives.

News Event	Acc. (R)	Acc. (N)
Charlie Hebdo	92.3%	21.8%
Ferguson	88.7%	34.5%
Germanwings crash	94.1%	19.2%
Ottawa shooting	90.6%	26.7%
Sydney Siege	86.9%	31.4%

Table 7: Performance of Qwen-Plus under basic prompts on PHEME.

C Model-agnostic Claim

We have conducted additional experiments using Qwen-Plus on the PHEME dataset to strengthen our model-agnostic claim. As shown in Table 8, our framework yields consistent improvements across baselines, with gains in accuracy (1-3%) and F1 (2-5% for R/N classes), closely mirroring the results obtained with DeepSeek-V3. We use the default temperature for both LLMs without

validation tuning: 1.0 for DeepSeek and 0.7 for Qwen-Plus. In practice, the default setting is sufficient and avoids introducing another decoding hyperparameter. These results confirm the framework's robustness across different LLM families.

Method	Acc	Prec	Rec	F1	Class
Bi-GCN	0.824	0.753/0.861	0.734/0.872	0.741/0.865	R/N
LLM-VN+Bi-GCN	**0.833**	0.752/0.880	0.778/0.858	0.767/0.864	R/N
RvNN	0.763	0.689/0.796	0.587/0.858	0.631/0.825	R/N
LLM-VN+RvNN	**0.786**	0.773/0.835	0.696/0.896	0.784/0.842	R/N
Graphsage	0.842	0.772/0.876	0.820/0.878	0.795/0.877	R/N
GSMA	0.848	0.834/0.856	0.823/0.851	0.840/0.860	R/N
LLM-VN+Graphsage	**0.852**	0.803/0.882	0.821/0.871	0.814/0.876	R/N
GAT	0.811	0.733/0.877	0.541/0.798	0.405/0.833	R/N
LLM-VN+GAT	**0.823**	0.730/0.880	0.780/0.850	0.750/0.860	R/N
HD-TRANs	0.766	0.656/0.783	0.697/0.755	0.676/0.768	R/N
LLM-VN+HD-TRANs	**0.779**	0.712/0.808	0.729/0.790	0.722/0.800	R/N
LINE	0.744	0.732/0.749	0.730/0.750	0.733/0.749	R/N
LLM-VN+LINE	**0.755**	0.702/0.782	0.727/0.769	0.720/0.776	R/N
DDGCN	0.855	0.877/0.831	0.763/0.892	0.816/0.860	R/N
LLM-VN+DDGCN	**0.865**	0.811/0.890	0.832/0.877	0.82/0.881	R/N

Table 8: Performance Comparison on PHEME Dataset via Qwen-Plus.

D Ablation Study

We conducted an ablation study of γ using Qwen-Plus + Bi-GAT on the PHEME dataset. The results in Table 9 that $\gamma = 0.2$ consistently yields the best performance, striking an optimal balance between incorporating global context and preserving local structural signals.

γ (%)	Acc.	Rec.	F1	Prec.	AUC
10	0.8138	0.7727	0.7401	0.7101	0.8767
15	0.8083	0.8021	0.7417	0.6897	0.8780
20	**0.8229**	**0.7754**	**0.7503**	**0.7268**	**0.8798**
25	0.8128	0.7888	0.7431	0.7024	0.8763
30	0.8018	0.8155	0.7385	0.6748	0.8744

Table 9: Ablation study on the virtual node mixing coefficient γ.

We further include a control experiment comparing two prediction formulations under the same backbone and training protocol. Specifically, we contrast (i) root-only classification, which predicts the rumor label directly from the root representation, with (ii) root-virtual link prediction, which predicts whether the root node is connected to the virtual node after virtual-edge augmentation. Since the encoder and data processing are kept identical, this comparison directly tests whether the link-prediction formulation is more effective than classifying from the root alone.

E Virtual-Node Analysis

We provide additional analysis of the proposed virtual-node augmentation from two perspectives. First, we quantify virtual-node connectivity by reporting the virtual-node degree distribution (unique neighbors) across PHEME events in Table 11. The p75 values indicate that, in typical cases, the virtual node connects to only a moderate number of nodes rather than forming a fully connected hub, while the max values highlight a small number of larger graphs where connectivity

Event	Root-only Acc.	Link Pred. Acc.
Charlie Hebdo	0.905	0.923
Ferguson	0.838	0.860
Germanwings Crash	0.867	0.894
Ottawa Shooting	0.871	0.876
Sydney Siege	0.801	0.829

Table 10: Root-only classification vs. root–virtual link prediction (accuracy).

Event	Split	p75	max	mean
Germanwings-crash	ALL	4	47	3.96
	Rumour	11	32	7.81
	Non-rumour	3	47	3.23
Ferguson	ALL	11	217	9.14
	Rumour	18	94	13.96
	Non-rumour	9	217	8.09
Ottawa-shooting	ALL	2	85	2.06
	Rumour	3	85	3.71
	Non-rumour	2	34	1.78
Sydney-siege	ALL	12	341	9.36
	Rumour	18	341	15.32
	Non-rumour	9	126	7.20
Charliehebdo	ALL	3	146	2.90
	Rumour	8	93	6.40
	Non-rumour	2	146	2.21

Table 11: Virtual-node degree statistics across PHEME events.

can be higher. Second, we examine whether the LLM-assigned subchain probability exhibits a systematic depth/length preference. We compute the Spearman rank correlation between node depth and the LLM-assigned probability in Table 12. The near-zero correlations suggest that LLM scores do not consistently favor shallow or deep nodes, supporting that our virtual-edge selection is not driven by a depth bias in the LLM scoring.

F Subchain Length and Token Statistics

On large dataset Weibo, we compute the average subchain length over all graphs and all nodes. To assess worst-case input size, we additionally identify the globally longest subchain across all 4,664 Weibo event graphs and count the number of tokens after concatenating the texts of all nodes along that path. As shown in Table 13, subchains are typically shallow. Our input length is far below the context limits of mainstream LLM APIs, for example, DeepSeek supports 128K tokens, while GPT, Qwen, and Gemini support about 1M tokens and Grok supports 2M. Truncation is not a practical concern in our setting.

Split	Spearman ρ (depth, prob)
All nodes	0.0285
Rumour (ref=0)	0.0060
Non-rumour (ref=1)	−0.0737

Table 12: Depth-bias analysis: correlation between node depth and LLM-assigned probability. Near-zero ρ indicates no systematic depth/length preference in LLM scores.

Statistic (Weibo)	Value
Avg. subchain length (#nodes)	2.687
Max. subchain length (#nodes)	73
Tokens on max subchain (concat.)	8385

Table 13: Weibo subchain length and token length statistics (globally longest subchain: 3907653488846171.json).

G Case Study on LLMs Performance

To illustrate the limitations of LLMs when performing rumour detection on the PHEME dataset with basic prompting, we present a representative example from the Charlie Hebdo shooting event.

- **Prompt to LLM:** "What was the Charlie Hebdo shooting?"
- **LLM Response:**

  > "On 7 January 2015, at about 11:30 a.m. in Paris, France, the employees of the French satirical weekly magazine *Charlie Hebdo* were targeted in a terrorist shooting attack by two French-born Algerian Muslim brothers, Saïd Kouachi and Chérif Kouachi. Armed with rifles and other weapons, the duo murdered 12 people and injured 11 others; they identified themselves as members of al-Qaeda in the Arabian Peninsula, which claimed responsibility for the attack. . . "

- **Source Tweet (from PHEME dataset):**

  > "10 people dead in Charlie Hebdo shooting in Paris."

- **Follow-up Prompt to LLM:** "Is this tweet spreading a rumor?"
- **LLM Response:**

  > "No, this refers to the real Charlie Hebdo terrorist attack that occurred on 7 January 2015 in Paris."

This case demonstrates that LLMs possess strong background knowledge of major historical events, accurately recalling key details such as the date, location, perpetrators, and official death toll (12 victims). However, when evaluating the specific tweet, the LLM overlooks a critical discrepancy: the tweet reports "10 people dead," reflecting an inaccurate early report that circulated during the breaking-news phase (initial media reports often cited 10 or 11 deaths before the confirmed toll of 12).

H Prompt Templates for LLM-based Rumor Scoring

This appendix presents the full prompt templates used in our two-stage LLM scoring procedure. We first construct a stable background knowledge base from the root post, and then use each root-to-node subchain to incrementally update the rumor probability under the same judgment basis.

Prompt A: Root Post Background-Knowledge Construction

```
You are evaluating the root post of a rumor-detection thread.

Your task is to build a stable background knowledge base and a reusable judgment
    basis for later subchain evaluation.
Do NOT overcommit to a final rumor decision at this stage.
First identify the root post's key factual claims, then compare them against
    general background knowledge.
If you are uncertain, explicitly preserve uncertainty instead of inventing details
    .

Important principles:
(1) Separate factual assessment from stylistic suspicion.
(2) Treat direct contradiction with known facts as stronger evidence than
    emotional tone.
(3) Treat unsupported new key claims as moderate risk rather than conclusive proof
     of rumor.
(4) Emotional or forwarding-inducing language alone is only weak evidence.
(5) If the post is broadly plausible but lacks verification, do not assign an
    overly confident non-rumor judgment.
(6) The output must define stable and reusable criteria that can be consistently
    applied to later subchains.
(7) Do not assume that general familiarity with the event is sufficient evidence
    that the post is true; evaluate the specific claims made in the post.

Please perform the following steps:

Step 1. Extract the key claims in the root post.
For each claim, identify:
- claim_id
- claim text
- claim type: event / casualty / time / place / entity / cause / other
- verifiability: high / medium / low

Step 2. Summarize the relevant background facts that are useful for evaluating
    these claims.
For each fact, identify:
- fact text
- relevance: high / medium / low
- certainty: high / medium / low

Step 3. Compare each extracted claim against the background facts.
For each claim, determine:
- relation_to_background: consistent / contradicted / unsupported / uncertain
- severity: integer from 0 to 3
- note: short explanation

Severity guidance:
```

```
- 3 = direct contradiction in key factual content (e.g., entity, time, place,
    number, or event outcome)
- 2 = important unsupported claim or suspiciously specific new factual addition
    without support
- 1 = weak stylistic or rhetorical signal only
- 0 = no apparent issue

Step 4. Identify style-based risk signals without letting them dominate factual
    evidence.
Report:
- emotional_language: 0 or 1
- urgency_or_forwarding: 0 or 1
- missing_source_for_key_claim: 0 or 1

Step 5. Produce an initial rumor prior score for the root post.
Use a float between 0 and 1:
- 0 means strongly non-rumor
- 1 means strongly rumor
- 0.5 means uncertain / insufficient evidence

Scoring rule:
- Start from uncertainty when evidence is limited.
- Move upward mainly for factual contradiction or important unsupported claims.
- Move downward only when the post is broadly consistent with background facts and
     does not contain serious factual conflict.
- Avoid extreme scores unless the evidence is strong.

Return STRICT JSON ONLY. Do not use markdown fences. Do not output any extra text.

Required JSON schema:
{
  "root_claims": [
    {
      "claim_id": "c1",
      "claim": "...",
      "claim_type": "event|casualty|time|place|entity|cause|other",
      "verifiability": "high|medium|low"
    }
  ],
  "background_facts": [
    {
      "fact": "...",
      "relevance": "high|medium|low",
      "certainty": "high|medium|low"
    }
  ],
  "claim_checks": [
    {
      "claim_id": "c1",
      "relation_to_background": "consistent|contradicted|unsupported|uncertain",
      "severity": 0,
      "note": "..."
    }
  ],
  "style_risk_signals": {
    "emotional_language": 0,
    "urgency_or_forwarding": 0,
    "missing_source_for_key_claim": 0
  },
```

```
  "root_prior_rumor_score": 0.5,
  "reasoning_basis": "A concise reusable judgment basis for future subchain
      scoring."
}

Root post:
[ROOT_POST]
```

Prompt B: Subchain-based Incremental Rumor Probability Update

```
You are evaluating a root-to-node subchain in a rumor-detection graph.

You must strictly follow the provided background knowledge base and reasoning
    basis.
Do NOT create a new evaluation standard.
Your task is to update the rumor probability of the ROOT POST using the replies in
     this subchain.

The subchain contains the root post followed by replies in temporal order.
Assess how the later replies affect the credibility of the root post.

Important principles:
(1) Start from the root_prior_rumor_score in the provided knowledge base.
(2) Distinguish factual correction from mere tone or agreement.
(3) Concrete evidence, contradiction, or correction matters more than emotion.
(4) Replies with no factual content should have little effect.
(5) If the evidence is mixed or weak, keep the updated score near the prior rather
     than becoming overconfident.
(6) Do not assume that a widely known event automatically validates the specific
    claims in the root post.
(7) Evaluate the replies as incremental evidence for or against the root post.

Before producing the final score, internally determine whether each non-root reply
     mainly functions as:
- support
- question
- deny
- correct
- add_evidence
- noise

Use these internal role judgments only to guide the probability update.
Do NOT output the role labels unless they are necessary for a brief explanation.

Update guidance:
- Increase rumor probability when a reply exposes factual inconsistency,
    fabricated detail, unsupported new claim, or source-related credibility
    problems.
- Decrease rumor probability when a reply provides concrete correction,
    clarification, authoritative attribution, or evidence that resolves an apparent
     mismatch.
- Pure agreement or pure denial without evidence should have only weak influence.
- Emotional reactions without factual content should have minimal influence.
- If the subchain adds no meaningful new evidence, the final score should remain
    close to the prior.

Final scoring rule:
```

```
- Begin from "root_prior_rumor_score".
- Apply only modest changes for weak or indirect evidence.
- Use larger changes only for concrete and relevant evidence.
- Keep the final score in [0, 1].
- Avoid extreme values unless the accumulated evidence is strong and consistent.

Return STRICT JSON ONLY. Do not use markdown fences. Do not output any extra text.

Required JSON schema:
{
  "probability": 0.5,
  "explanation": "A brief explanation of how the subchain updates the root rumor
      score."
}

Background knowledge base:
[KNOWLEDGE_BASE]

Subchain text:
[SUBCHAIN]
```

Design Rationale

- We first construct a stable prior judgment basis from the root post, then reuse it across all subchains, so that different subchains are scored under a consistent standard.
- We explicitly separate factual contradiction, unsupported claims, and weak stylistic signals, preventing superficial rhetoric from dominating the judgment.
- We model subchain scoring as an incremental update from a prior score, rather than an independent re-decision for each subchain.
- We require the model to internally distinguish the role of each reply (e.g., support, correction, evidence addition, or noise), but only output the final probability and a brief explanation, which reduces token cost while preserving role-aware reasoning.
- We preserve uncertainty when evidence is weak, improving score stability before thresholding for virtual-edge construction.